Product Sentiment Analyzer: Scraping, Scoring, and Visualizing E-Commerce Reviews on Demand

Meetha Dinesan(156)
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College
Chennai, India
meethadinesan.2024.aids@rajalakshmi.edu.in

Nandana Pramod(180)
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College
Chennai, India
nandanapramod.2024.aids@rajalakshmi.edu.in

***Abstract*—Deciding whether a product is any good often means reading dozens of reviews, and most shoppers do not. We built Product Sentiment Analyzer to do that reading. Given a product name, the system scrapes reviews from Flipkart with a headless browser, cleans the text, scores each review with the Valence Aware Dictionary and sEntiment Reasoner (VADER) lexicon, stores the results in MongoDB, and shows the sentiment split and the individual reviews on a React dashboard. When a product has not been seen before, the Flask back end scrapes it in a background thread while the front end polls for progress, so the page does not hang. A live query for "iPhone 15" returned 25 reviews, 84% of them positive. For a quantitative check we used 55 Puma shoe reviews from the offline pipeline: 85.5% were labelled positive, and the labels matched star-rating-derived labels 87.3% of the time (95% bootstrap interval 78.2% to 94.5%), slightly ahead of a TextBlob baseline at 83.6%, although the gap is only two reviews. Moving the neutral threshold from 0.05 to 0.1 changed nothing, and skipping our text cleaning changed three scores but no labels. We also describe where the method fails: misspellings, mixed opinions, and defect reports that contain no emotional words. The application is containerized and runs on Render and Vercel.**

***Index Terms*—Flask, MongoDB, opinion mining, product reviews, React, Selenium, sentiment analysis, VADER, web scraping.**

I. Introduction

Shoppers on large e-commerce sites usually decide using the star average and whichever reviews they happen to see first. That is a thin basis for a purchase, yet reviews clearly matter. They have been shown to move sales [1], and what makes a review helpful has become a research question of its own [2]. A popular product can collect thousands of reviews, and nobody reads them all.

Sentiment analysis tries to close that gap by labelling the opinion expressed in a piece of text as positive, negative, or neutral [3], [4]. There are two broad ways to do it. One is to train a classifier on labelled text, which today usually means fine-tuning a pre-trained transformer such as Bidirectional Encoder Representations from Transformers (BERT) [5]. The other is to score text against a hand-built lexicon, as the Valence Aware Dictionary and sEntiment Reasoner (VADER) does [6]. The first route is more accurate on benchmarks but needs labelled data, training, and a model-serving setup. The second needs none of those and scores a review in well under a second on an ordinary central processing unit (CPU). For a project whose data are short, unlabelled reviews scraped at request time, and whose target is a small cloud instance, we chose the second.

The harder problem turned out to be getting the reviews at all. A fixed dataset cannot answer "what are people saying about this particular shoe right now?", so the system has to fetch reviews when a user asks. Driving a real browser through a search, a product page, and a scrolling review list is slow. The fixed waits in our scraper alone can add up to well over a minute for five products, before any page-load time is counted, which is far too long to hold a Hypertext Transfer Protocol (HTTP) request open. We therefore made scraping an asynchronous job: the application programming interface (API) accepts the request, returns immediately, and lets the client poll.

This paper describes the resulting system and evaluates it on a real sample. Our contributions are (i) an end-to-end pipeline from scraping to dashboard, (ii) the background-job and polling workflow that makes on-demand scraping usable from a browser, (iii) a representational state transfer (REST) API that exposes scored reviews and aggregated sentiment, and (iv) an evaluation that compares VADER's labels with star ratings, a TextBlob baseline, the effect of our preprocessing, and the sensitivity to the neutral threshold, followed by an error analysis.

Section II reviews related work, Section III presents the design, and Section IV explains the methods in detail. Section V covers implementation and deployment, Section VI reports the evaluation, Section VII lists limitations and future work, and Section VIII concludes.

II. Related Work

A. Sentiment Classification

Early work treated sentiment as ordinary text classification. Pang et al. [7] trained Naive Bayes, maximum-entropy, and support vector machine (SVM) classifiers on movie reviews and found that bag-of-words features worked surprisingly well. Larger corpora followed, including the IMDb dataset of Maas et al. [8], which also served as a testbed for learned word vectors. Neural models came next: convolutional networks over word embeddings [9] and recursive networks trained on a phrase-level sentiment treebank [10] both improved on the older feature-based methods, and pre-trained transformers [5] now dominate the benchmarks. Surveys by Pang and Lee [3], Liu [4], and Medhat et al. [11] cover this progression in more detail.

Lexicon-based methods took a different path. Taboada et al. [12] showed that a lexicon enriched with rules for negation and intensification is competitive and, importantly, easier to adapt across domains because it does not depend on training data. VADER [6] follows the same idea but was tuned for short, informal text. Its lexicon was rated by human annotators, and its rules handle punctuation emphasis, capitalization, degree adverbs, negation, and the conjunction "but". Ribeiro et al. [13] compared a large number of off-the-shelf methods across many labelled datasets and found that no method wins everywhere, though VADER was among those that performed consistently well. That finding fits our situation: reviews are short, informal, and unlabelled.

B. Mining Product Reviews

Hu and Liu [14] were among the first to mine customer reviews for the product features people comment on and the polarity of each comment. Their aspect-level view is richer than the single label per review that we produce, and we return to it in Section VII. Most published work in this area, though, evaluates a classifier on a fixed corpus. The practical questions of how fresh reviews get collected, how long that takes, and how a user sees the result receive less attention. Those questions are the focus of this paper.

C. Web Data Acquisition

Review text is usually reachable only through a site's pages, because many platforms offer no open review interface. The simplest way to collect it is to download the HTML and parse it. That fails when a page builds its content with JavaScript [15], which is how Flipkart's review list behaves when the visitor scrolls. Browser-automation tools such as Selenium sidestep the problem by driving a real browser, at the price of speed and of extra moving parts: a browser binary, a matching driver, and selectors that break when the markup changes. We accept that price, and it is the reason scraping is a background job in our design.

III. System Design

A. Requirements

We set a short list of requirements before writing code. A user should be able to type any product name and get a sentiment summary plus the individual reviews. Products the system has never seen should be fetched automatically. The interface must stay responsive while that happens. Nothing should need a graphics processing unit (GPU) or a trained model file, so that the whole system fits on a small cloud instance. Everything else, such as authentication or multi-site support, was left out on purpose.

B. Architecture

The system has three tiers (Fig. 1). A React single-page application forms the presentation tier. The application tier is a REST [16] API written with Flask [17] and split into three service modules for scraping, sentiment scoring, and database access. Reviews live in a MongoDB Atlas cluster [18]. The scraper service drives headless Chromium through Selenium to read Flipkart's pages [15].

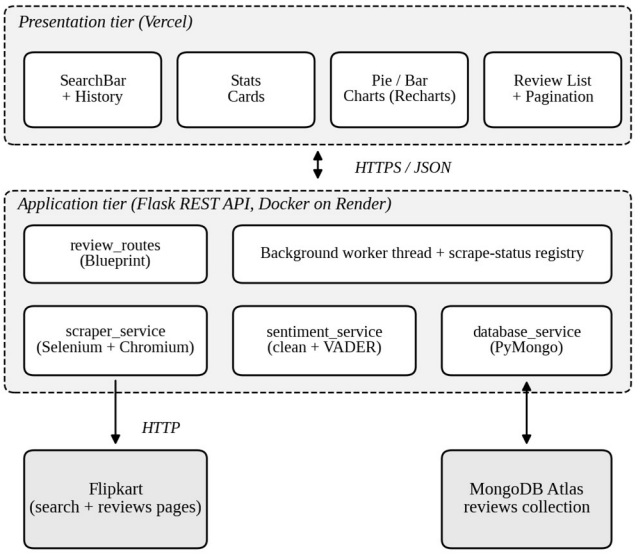


Fig. 1. Three-tier architecture of Product Sentiment Analyzer.

C. Technology Stack

Table I lists what each layer uses, where NLTK is the Natural Language Toolkit. Version numbers come from the project's dependency files.

TABLE I
Technology Stack

Layer	Technology	Role
Front end	React 19, Vite 8, Axios, Recharts 3	Interface, API calls, charts
Back end	Python 3.11, Flask 3.1, Flask-CORS	REST API
Acquisition	Selenium 4.45, Chromium	Headless scraping
NLP	NLTK 3.9 (VADER), pandas 3	Cleaning, scoring
Database	MongoDB Atlas, PyMongo 4.17	Persistence
Deployment	Docker, Render, Vercel	Hosting

D. Request Flow

Fig. 2 traces one search. The client first asks the API for stored reviews and the sentiment summary. If the product exists, the dashboard renders at once. If either request fails, which for an unseen product means a not-found error, the client takes the product to be missing and posts a scrape request. The server replies with HTTP 202 and starts a background thread that scrapes, scores, and stores the reviews, then marks the product as done. Meanwhile the client checks the status endpoint every ten seconds and draws the dashboard as soon as it reads done.

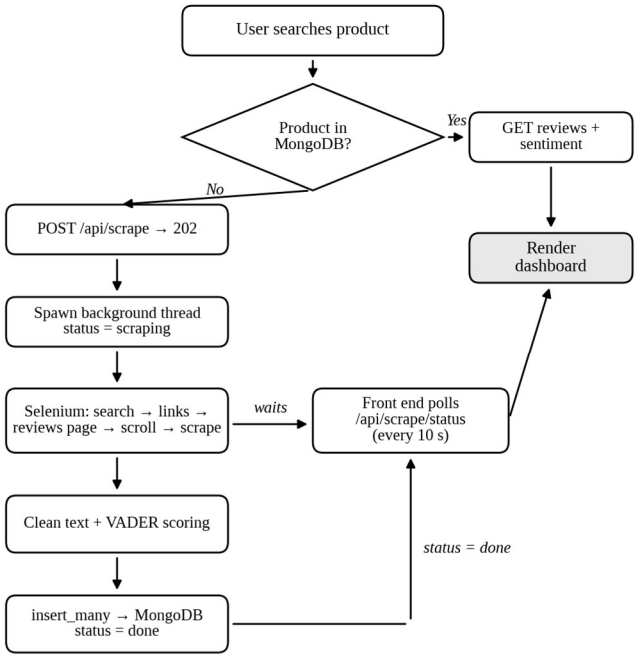


Fig. 2. On-demand search and scraping workflow.

IV. Methodology

A. Data Acquisition

The scraper opens Flipkart in headless Chrome, closes the login pop-up if one appears, types the query into the search box, and collects product links from the results. Flipkart uses different cascading style sheet (CSS) class names in different product categories, so the scraper tries a list of known class names first. If none match, it falls back to every anchor whose address contains `/p/`. For each product it builds the review-page address from the product slug, item identifier, and `pid` parameter. It then scrolls up to five times, pausing two seconds after each scroll, and stops early once ten review cards are visible or three consecutive scrolls bring nothing new. The rating, title, and body of each card are read from the page, and cards without a body are skipped.

The browser runs with `--headless=new`, `--no-sandbox`, `--disable-dev-shm-usage`, and `--disable-gpu`, plus flags that hide Selenium's automation markers. At start-up the driver checks for the `CHROME_BIN` and `CHROMEDRIVER_PATH` environment variables. If they are set, as in the Docker image, it uses the system Chromium. Otherwise it fetches a matching driver with webdriver-manager, which is convenient on a development laptop. The API path visits at most five products and reads at most ten reviews from each, so one query yields up to 50 reviews. The command-line pipeline we used for offline experiments visits up to ten products.

B. Preprocessing

The offline pipeline is a chain of four stages that each read and write a comma-separated values (CSV) file (Table II). It drops duplicate reviews (same body text), fills a missing title with "No Title" and a missing rating with 0.0, and then cleans both text fields. Cleaning strips uniform resource locators (URLs) and Hypertext Markup Language (HTML) tags, replaces every character other than letters, digits, whitespace, and the marks . , ! ? with a space, collapses runs of whitespace, and lower-cases the result. Rows whose body ends up empty are removed. The API path uses the same cleaning function on the text it scores but keeps the original text for display, so users see reviews as written. Products loaded through the offline pipeline are stored with the cleaned text instead, and deduplication is only done there.

TABLE II
Offline Pipeline Stages

Stage	Script	Output	Rows
Scrape	scrape_reviews.py	raw_reviews.csv	64
Clean	clean_data.py	cleaned_reviews.csv	55
Score	sentiment_analysis.py	sentiment_results.csv	55
Load	database.py	MongoDB reviews	–

C. Sentiment Scoring

Each review body is scored with the VADER analyzer that ships with NLTK [6], [19]. VADER adds up the valence of every lexicon word it finds, adjusts the values with its rules, and squashes the sum x into a compound score C between −1 and 1:

$$C = x / \sqrt{(x^2 + \alpha)} \quad (1)$$

Here $\alpha = 15$ is the normalization constant used by VADER. We label a review *Positive* if $C \geq 0.05$, *Negative* if $C \leq -0.05$, and *Neutral* otherwise, which are the thresholds recommended by the VADER authors. Only the compound score and the label are stored.

D. Storage

Reviews go into a `reviews` collection. Each review is one document with the fields in Table III. The sentiment summary is computed in the database with an aggregation pipeline that matches on the product and groups by label, and the API turns the counts into percentages with one decimal place. Products are matched by exact string, a simplification we revisit in Section VII.

TABLE III
Review Document Schema

Field	Type	Content
_id	ObjectId	Unique key generated by MongoDB
rating	string/number	Star rating, parsed by the client
title	string	Review headline
body	string	Review text (original on the API path)
sentiment_score	number	VADER compound score
sentiment_label	string	Positive, Neutral, or Negative
product	string	Query used to fetch the review

E. REST API and Background Jobs

Table IV lists the endpoints. The scrape endpoint behaves differently depending on state: it returns immediately if the product is already stored, reports progress if a job is running, and otherwise starts a daemon thread and answers 202. An in-memory dictionary records each product as scraping, done, or

error, and the status endpoint reports done as soon as stored reviews exist. Algorithm 1 gives the job itself.

TABLE IV
REST API ENDPOINTS

Endpoint	Method	Purpose
/api/health	GET	Liveness check
/api/products	GET	List stored products
/api/reviews	GET	Scored reviews of a product
/api/sentiment	GET	Sentiment counts and percentages
/api/scrape	POST	Start background scraping (202)
/api/scrape/status	GET	Scraping state of a product

Algorithm 1. Background job for an unseen product

```
procedure ScrapeJob(p)
  status[p] ← scraping
  R ← RunScraper(p, max products = 5, max reviews = 10)
  if R is empty then status[p] ← error; return
  for each review r in R do
    C ← VADER(Clean(r.body))
    r.score ← C; r.label ← Label(C); r.product ← p
  InsertMany(R); status[p] ← done
  on exception: status[p] ← error
```

Several requests can reach the server at once. Each scrape runs in its own thread with its own browser instance. The status dictionary is shared between those threads and the request handlers, and simple assignments to a dictionary are atomic in CPython, so the code uses no lock. The same product cannot be scraped twice at the same time, because a second request sees the scraping state and returns immediately. Different products can be scraped in parallel, however, and every job starts a full Chromium instance, so memory use grows with the number of concurrent jobs.

F. Front End

The interface is a Vite-built React app that talks to the API through Axios. A search bar takes the product name, and a small history component remembers the last five distinct searches. Four summary cards show the number of reviews, the average star rating, the share of positive reviews, and the dominant sentiment. The average is computed in the browser from the ratings that parse to a positive number, so reviews with a missing rating do not pull it down. A pie chart and a bar chart, both drawn with Recharts, show the label distribution. Below them, reviews appear as cards with the star rating, a colour-coded sentiment badge, the title, the text, and the compound score, five to a page. While a new product is being scraped, a spinner and a status banner (first "Product not found. Scraping in background… please wait.", later "Still scraping… please wait.") keep the user informed.

V. IMPLEMENTATION AND DEPLOYMENT

The back end ships as a Docker image built on `python:3.11-slim`. The image installs Chromium and its driver from the distribution packages, sets the two environment variables the scraper looks for, installs the Python dependencies, copies in both the API and the scraper code, and starts Flask on all network interfaces so the hosting platform can reach it. The container runs on Render, the React build is served from Vercel, and the database is a MongoDB Atlas cluster. Because the front end and the API live on different origins, the API enables cross-origin requests with Flask-CORS. The connection string and port come from environment variables rather than the source code.

The code is organized so that each stage can be run alone. The scraping, cleaning, scoring, and database-loading scripts all work from the command line, and a fifth script chains them for a new product. We used this to test every stage before it was wired into the API. The experiments in Section VI were run on the files that this offline pipeline writes: the raw, cleaned, and scored reviews.

A few details were more awkward than the architecture suggests. Review cards on the page are identified by generated class names, and the star rating is found by matching inline colour styles, so both are tied to Flipkart's current markup and will need updating when it changes. The fallback to `/p/` links exists for the same reason. Polling every ten seconds is a blunt instrument, but it needs no extra infrastructure, and a job that takes a minute does not benefit from anything faster. Figs. 3 to 6 are screenshots from a local run of the finished system for the query "iPhone 15".

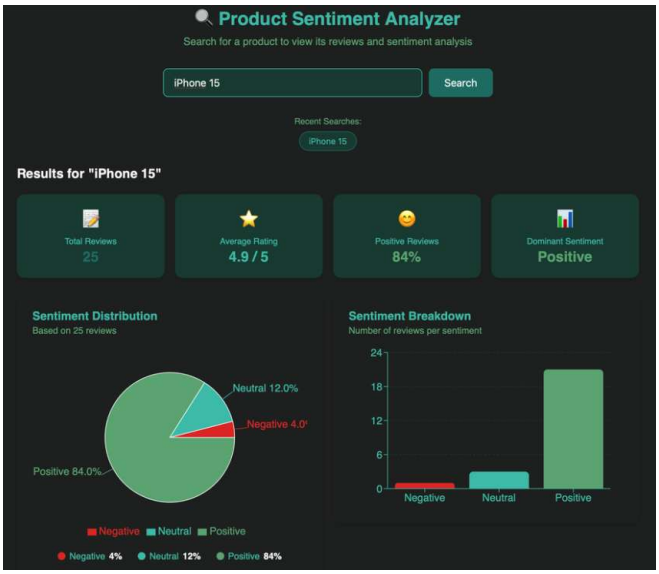


Fig. 3. Dashboard for the query "iPhone 15": summary cards, pie chart, and bar chart.

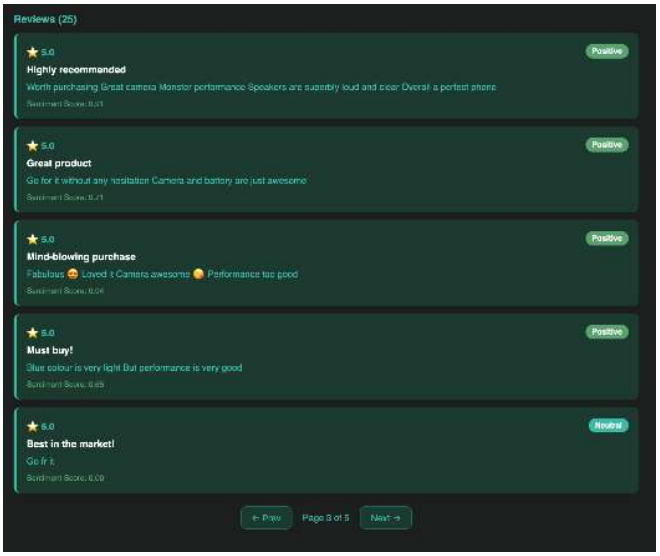


Fig. 4. Review list (page 3 of 5) with star ratings, sentiment badges, and compound scores.

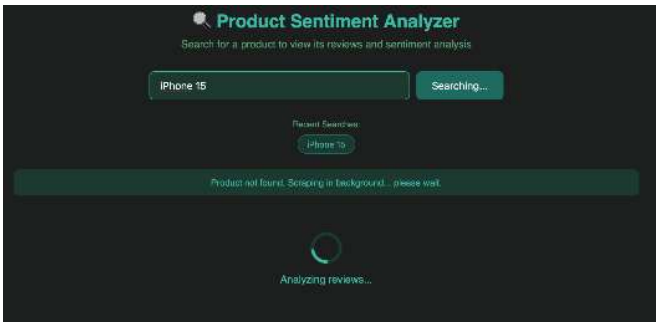


Fig. 5. Interface while a product that is not yet stored is being scraped.

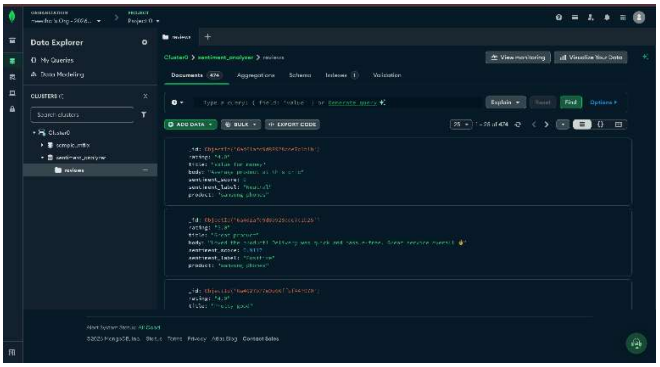


Fig. 6. MongoDB Atlas view of the reviews collection (474 documents).

VI. Evaluation

A. Data and Setup

We used two sources of data. The first is an offline run of the command-line pipeline for a Puma sports-shoes search. It returned 64 reviews. Nine had the same body text as an earlier review and were dropped, leaving 55, and none became empty after cleaning. All quantitative results below use these 55 reviews, because they were exported to CSV and can be re-scored. The second source is the deployed system itself, which we queried live for "iPhone 15" (Section VI-B). Table V summarizes both, with a dash where the dashboard does not show a value. The Puma reviews are very short, with a median of three words, and two thirds of them carry five stars, a skew that is common for online reviews. We could not hand-label the reviews, so the star rating serves as a weak reference throughout: four or five stars count as positive, three as neutral, and one or two as negative. All scores below are from NLTK's VADER, and re-scoring the stored text reproduced the saved scores exactly.

TABLE V
Case-Study Datasets

Statistic	Offline (Puma)	Live (iPhone 15)
Raw reviews scraped	64	–
Duplicates removed	9	–
Reviews analysed	55	25
Words per review (mean / median / max)	6.4 / 3 / 40	–
5 / 4 / 3 / 1 stars	36 / 15 / 3 / 1	–
Mean star rating	4.5	4.9
Positive / neutral / negative	47 / 4 / 4	21 / 3 / 1

B. Live Example

Figs. 3 and 4 show the system answering the query "iPhone 15" on a local run. The dashboard reports 25 reviews with an average rating of 4.9 out of 5. VADER labelled 84% of the reviews positive, 12% neutral, and 4% negative, which is 21, 3, and 1 reviews. The list shows five reviews per page, so the 25 reviews fill five pages, and each card carries the star rating, the sentiment badge, and the compound score. For this query the product was not yet stored when it was first searched, which is why the interface displayed the scraping banner of Fig. 5 until the job finished. The Atlas view in Fig. 6 shows a reviews collection of 474 documents, which includes reviews stored for other queries. These live results illustrate the workflow, but they are not used for the accuracy figures below, since the stored reviews were not exported.

C. Sentiment Distribution

VADER labelled 47 reviews (85.5%) as positive, 4 (7.3%) as neutral, and 4 (7.3%) as negative (Fig. 7). The mean compound score was 0.493 with a median of 0.625 and a standard deviation of 0.389. The histogram in Fig. 8 leans heavily to the right, matching the star ratings. It also shows a small spike at exactly zero. Those are reviews in which the lexicon found no polar word at all.

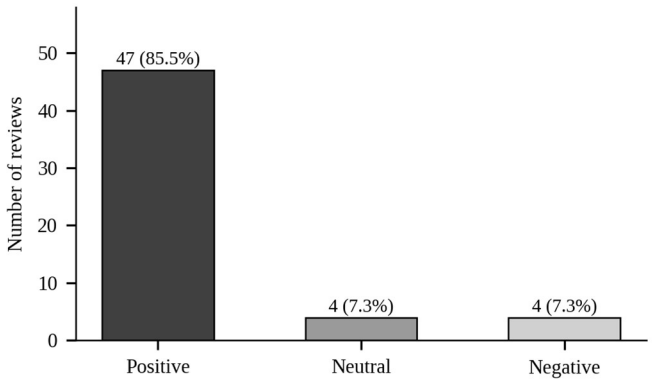


Fig. 7. Sentiment label distribution (n = 55).

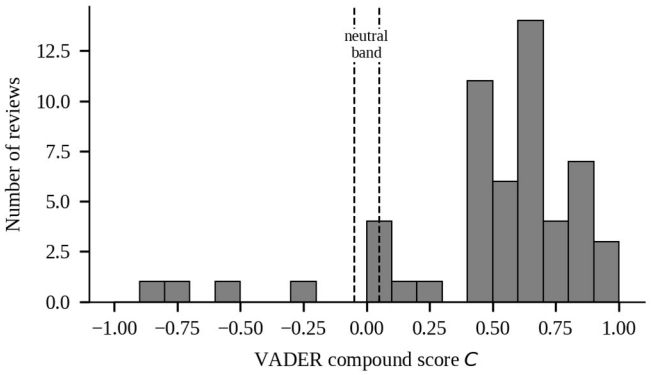


Fig. 8. Histogram of VADER compound scores. Dashed lines mark the ±0.05 thresholds.

D. Agreement with Star Ratings

Accuracy against the star-derived labels, defined in (2) over $N = 55$ reviews, is 87.3%, which is 48 correct labels. A bootstrap over reviews (5000 resamples) gives a 95% interval of 78.2% to 94.5%, so the true figure for this kind of data could plausibly be anywhere in that range. If we only ask whether a review is positive or not, accuracy rises to 92.7%. The Spearman correlation between the compound score and the star rating is 0.40 ($p = 0.003$), a moderate relationship that Fig. 9 and Table VI make visible: the mean score climbs from −0.26 for three-star reviews to 0.61 for five-star reviews. The one-star review sits at 0.00 and is discussed below.

$$Acc = (1/N)\ \Sigma_{i=1\ldots N}\ I(\hat{y}_i = y_i) \quad (2)$$

TABLE VI
MEAN COMPOUND SCORE BY STAR RATING

Stars	Reviews	Mean compound score
5	36	0.607
4	15	0.402
3	3	−0.256
1	1	0.000

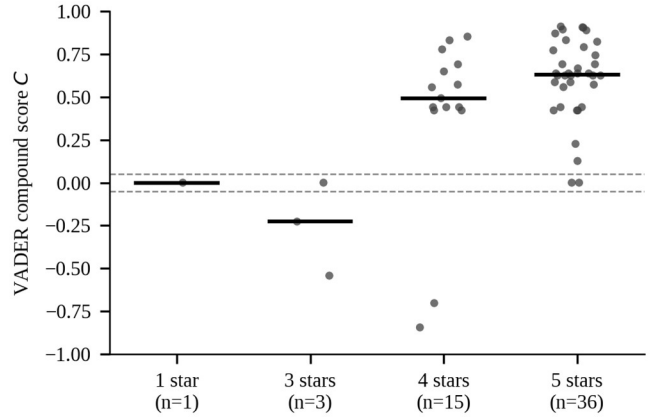


Fig. 9. Compound score of every review by star rating. Horizontal bars mark the median, dashed lines the ±0.05 thresholds.

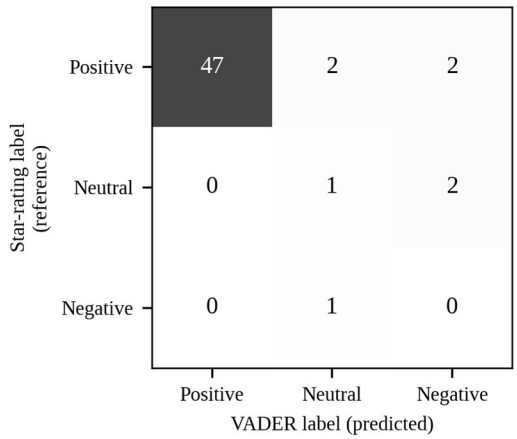


Fig. 10. Confusion matrix of VADER labels against star-rating labels.

TABLE VII
PER-CLASS AGREEMENT WITH STAR-RATING LABELS

Class	Precision	Recall	Reference n
Positive	100.0%	92.2%	51
Neutral	25.0%	33.3%	3
Negative	0.0%	0.0%	1

The positive class carries the result (Fig. 10). Every review VADER called positive has four or five stars, and 47 of the 51 reference-positive reviews were found. The neutral and negative reference classes hold three reviews and one review, so the percentages in Table VII for those rows say very little and are shown only for completeness.

E. Comparison with a Baseline

To check that VADER earns its place, we scored the same reviews with TextBlob [20], whose default analyzer is also lexicon-based, using the same ±0.05 cut-offs. Table VIII shows VADER ahead on all three measures. The accuracy gap is two reviews, though, and the bootstrap interval above is far wider than that, so we read the comparison as "at least as good" rather than "clearly better". The larger difference is in correlation with star rating, 0.40 against 0.20.

TABLE VIII
VADER VERSUS TEXTBLOB

Method	3-class acc.	Positive vs. rest	Spearman ρ
VADER	87.3%	92.7%	0.40
TextBlob	83.6%	85.5%	0.20

F. Effect of Text Cleaning

Our cleaning step lower-cases text and removes emoticons, emoji, and most punctuation, all of which VADER can use [6]. We therefore scored the original, uncleaned text as well. Only three of the 55 scores changed, no label changed, and accuracy stayed at 87.3%. Table IX lists the three reviews. Removing the smiley from the second one cut its score from 0.60 to 0.23. In the third, cleaning pushed the score up, from 0.36 to 0.74, so the effect is not even consistently in one direction. On this sample the cleaning is harmless for labels, but reviews with more emoticons or emphatic capitals could behave differently.

TABLE IX
REVIEWS WHOSE SCORE CHANGED AFTER CLEANING

Review (abridged)	Stars	Original	Cleaned
Nice shoes for Cricket. So comfortable … Must buy. *****	5	0.858	0.870
Yeah i like it … hope this puma shoes survive my training stress :)	5	0.599	0.226
Just looking like of Wow (three emoji)	5	0.361	0.743

G. Neutral Threshold

Fig. 11 varies the neutral threshold τ from 0.05 to 0.5. Between 0.05 and 0.1 nothing changes. At 0.2 and above, accuracy slips to 85.5%, and at 0.5 it falls to 65.5% because 18 reviews, a third of the sample, land in the neutral band. The reason is that short reviews rarely score high: a quarter of all reviews have a compound score of 0.42 or lower even though 85% are positive. A stricter threshold would therefore label many plainly positive reviews as neutral, and we keep the standard 0.05.

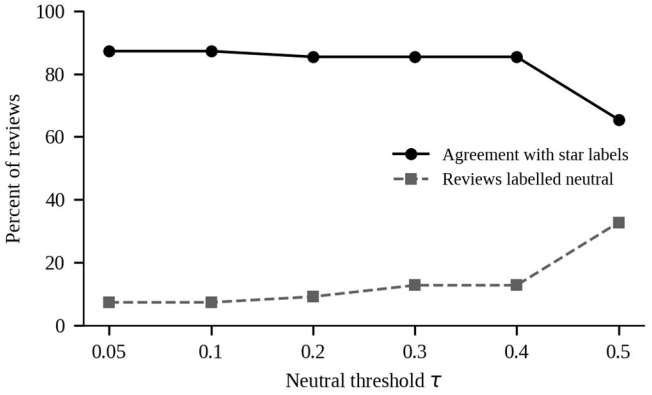


Fig. 11. Effect of the neutral threshold τ on agreement with star labels and on the share of neutral reviews.

H. Review Length

Agreement also varies with review length (Table X). Reviews of one to three words make up about half the sample and are matched 89.3% of the time, but they account for three of the four neutral labels. All four neutral labels are scores of exactly 0.0, cases where the lexicon found nothing to score. Reviews of four to eight words are all matched. Agreement is lowest, at 75.0%, for reviews of nine or more words, which is where mixed opinions and complaints inside otherwise praising text appear. The three groups hold 28, 11, and 16 reviews, so the differences are suggestive rather than conclusive.

TABLE X
AGREEMENT BY REVIEW LENGTH

Words	Reviews	Agreement	Neutral labels
1–3	28	89.3%	3
4–8	11	100.0%	0
9 or more	16	75.0%	1

I. Error Analysis

Seven reviews disagree with their star label. Four of the 55 reviews score exactly 0.0, and three of those are in the error set. Two are five-star reviews, “mind blowing product” and a misspelled “nise product”, where the lexicon simply has no entry. The third is the single one-star review, which reports a cracked sole capsule that fills with water. It describes a defect but contains no sentiment word, and VADER cannot infer displeasure from facts. The live example shows the same effect: in Fig. 4, a five-star review that reads “Go fr it” is shown as neutral with a score of 0.00, because the misspelled word is not in the lexicon.

Two other errors come from mixed opinions. “Product is good but packing is very bad” has four stars but scores −0.70, because VADER gives extra weight to the clause after “but”. The review reads as mixed, and the star rating reflects the product, not the packaging. A second four-star review describes a slippery sole that caused an injury and scores −0.84. Its text is plainly negative, so this looks like an inconsistent star rating, not a classifier error. The last two cases are three-star reviews, “vary bad quality” (−0.54) and “look wise shoes are ok, but they are not comfortable to wear all day” (−0.23). Both are labelled negative while three stars were mapped to neutral. In effect, a three-star rating is ambiguous, and the text leans negative in both cases.

These cases suggest that part of the 12.7% disagreement is noise in the reference, not in VADER. A hand-labelled test set would settle it, and we did not have one.

J. Discussion

Taken together, the results suggest that VADER is a sensible default for this kind of data and an unreliable judge of individual edge cases. The aggregate picture is stable: the label shares did not move when we changed the cleaning step, and they barely moved across a sensible range of thresholds. For the dashboard’s main job, telling a shopper whether the mood around a product is mostly positive or mostly negative, that is enough. For a verdict on a single review it is not, which is why each review card shows the original text and the compound score next to the badge, so that a borderline label can be judged by eye.

The comparison with star ratings has a second use that we did not plan for. Reviews whose text and rating disagree, like the injured four-star reviewer above, are the ones a seller or moderator would most want to read, and the stored scores make them easy to find. We have not built that feature, but the data to support it are already in the database.

K. Threats to Validity

Four points limit how far the numbers can be pushed. On construct validity, agreement with a star rating is only a proxy for sentiment, and a reviewer can give four stars while criticizing the packaging, as one review in our sample does. On internal validity, the ±0.05 thresholds come from the VADER authors and were not tuned on our data, but the threshold sweep and the cleaning check were run on the same 55 reviews as the headline numbers, so they are descriptive and not held-out results. On external validity, the sample comes from one site, one search, and the first ten review cards of each product, and the language is English with some code-mixing. Finally, with

55 reviews the 95% interval for accuracy spans more than 16 points, and the neutral and negative reference classes contain three and one reviews.

VII. Limitations and Future Work

The evaluation is small. It covers 55 reviews from one search, and it measures agreement with star ratings, not with human judgement. The deployed database holds more reviews than that, but only the 55 exported ones could be re-scored. The sample also contains only the first ten review cards of each product, so it reflects whatever ordering the site shows by default, and that may favour positive or popular reviews.

Several weaknesses are specific to the implementation. The scraper depends on generated class names and inline styles that the site can change without notice, and heavy scraping may conflict with the site's terms of use. VADER's lexicon is English, so code-mixed text and misspellings are scored poorly, as the error analysis showed. Only the review body is analysed, and the title and rating are ignored. Products are matched by exact name, so "puma shoes" and "Puma Shoes" are two different entries. Job status is held in the memory of one process and disappears on restart, and the database client relaxes Transport Layer Security (TLS) certificate validation to suit the hosting environment, which should be tightened before real use. The client also treats any failed lookup as a missing product, so a network error can trigger an unnecessary scrape.

On data handling, the scraper stores only the rating, title, and body of each review. It does not collect reviewer names or profile links, so no personal identifiers enter the database. Scraping a commercial site still raises questions about its terms of use. A deployment beyond coursework would need permission or an official data API.

The most useful next step is a hand-labelled set of a few hundred reviews, which would allow a proper comparison between VADER, other lexicons, and a fine-tuned transformer [5]. Beyond that, aspect-level analysis in the spirit of Hu and Liu [14] would let the dashboard say that comfort is praised while packaging is not, instead of giving one score per review. Smaller improvements include normalizing product queries, moving scraping to a persistent task queue, refreshing stored reviews periodically, and supporting more sites and languages.

VIII. Conclusion

We built and evaluated a web application that fetches product reviews on demand, scores them with VADER, and presents the result on an interactive dashboard. Treating scraping as a background job with client-side polling was the design decision that made the rest practical, since a full scrape takes far longer than a browser will wait for a single response. On the 55 offline reviews the labels agreed with star-derived labels 87.3% of the time, which is respectable for a method that needs no training, and the error analysis shows where it falls short: unfamiliar words, mixed opinions, and unemotional defect reports. Better evaluation data and aspect-level analysis are the next steps.

References

[1] J. A. Chevalier and D. Mayzlin, "The effect of word of mouth on sales: Online book reviews," *J. Marketing Res.*, vol. 43, no. 3, pp. 345–354, Aug. 2006.

[2] S. M. Mudambi and D. Schuff, "What makes a helpful online review? A study of customer reviews on Amazon.com," *MIS Q.*, vol. 34, no. 1, pp. 185–200, Mar. 2010.

[3] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Found. Trends Inf. Retr.*, vol. 2, no. 1–2, pp. 1–135, 2008.

[4] B. Liu, *Sentiment Analysis and Opinion Mining*. San Rafael, CA, USA: Morgan & Claypool, 2012.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[6] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. 8th Int. AAAI Conf. Weblogs Social Media (ICWSM)*, Ann Arbor, MI, USA, 2014, pp. 216–225.

[7] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Philadelphia, PA, USA, 2002, pp. 79–86.

[8] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Portland, OR, USA, 2011, pp. 142–150.

[9] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Doha, Qatar, 2014, pp. 1746–1751.

[10] R. Socher *et al.*, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. EMNLP*, Seattle, WA, USA, 2013, pp. 1631–1642.

[11] W. Medhat, A. Hassan, and H. Korashy, "Sentiment analysis algorithms and applications: A survey," *Ain Shams Eng. J.*, vol. 5, no. 4, pp. 1093–1113, Dec. 2014.

[12] M. Taboada, J. Brooke, M. Tofiloski, K. Voll, and M. Stede, "Lexicon-based methods for sentiment analysis," *Comput. Linguist.*, vol. 37, no. 2, pp. 267–307, Jun. 2011.

[13] F. N. Ribeiro, M. Araújo, P. Gonçalves, M. A. Gonçalves, and F. Benevenuto, "SentiBench: A benchmark comparison of state-of-the-practice sentiment analysis methods," *EPJ Data Sci.*, vol. 5, art. no. 23, 2016.

[14] M. Hu and B. Liu, "Mining and summarizing customer reviews," in *Proc. 10th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Seattle, WA, USA, 2004, pp. 168–177.

[15] R. Mitchell, *Web Scraping with Python*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.

[16] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[17] M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.

[18] K. Chodorow, *MongoDB: The Definitive Guide*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2013.

[19] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.

[20] S. Loria, "TextBlob: Simplified text processing," 2018. [Online]. Available: https://textblob.readthedocs.io